Компактная русскоязычная визуально-языковая модель на открытых данных VK

Qwen3-VL-2B-Instruct + LoRA, дообученная на датасетах deepvk на одной GPU 8 ГБ · Засимов Георгий Алексеевич · октябрь 2026 · код и адаптер: github.com/geozasimov/vk_vlm_project

Кратко

Дообучили компактную VLM Qwen3-VL-2B-Instruct (2.1 млрд параметров) методом LoRA на открытых данных VK: GQA-ru и LLaVA-Instruct-ru. Обучение шло 38 минут на ноутбучной RTX 5060 с 8 ГБ памяти. На бенчмарке **GQA-ru** точность выросла с 41.1 до **52.5** ExactMatch (+11.4 п.п.); это выше обеих открытых моделей VK (llava-gemma-2b-lora — 46.4, llava-saiga-8b — 51.4). На **MMBench-ru** (CircularEval) модель показывает **68.7** против 66.5 у исходной модели и 56.7 у llava-saiga-8b. Прирост на MMBench-ru в основном связан с форматом ответа (буква вместо рассуждения), поэтому его корректнее читать как «общие навыки не пострадали» (см. раздел 5).

Модель	Параметры	GQA-ru, EM	MMBench-ru, Circular
deepvk/llava-gemma-2b-lora	3B	46.4	40.2
deepvk/llava-saiga-8b	8B	51.4	56.7
Qwen3-VL-2B-Instruct (без дообучения)	2B	41.1	66.5
Qwen3-VL-2B + LoRA (наша)	**2B**	**52.5**	**68.7**

> Метрики моделей VK взяты из их карточек на Hugging Face (lmms-eval, MMBench без GPT-судьи). Наши метрики посчитаны собственными скриптами по тому же протоколу (промпты, нормализация, CircularEval). Подробности и оговорки — в разделе 5.

1. Описание проекта

Актуальность

Визуально-языковые модели (VLM) принимают изображение и текст и отвечают в свободной форме: описывают сцену, отвечают на вопросы, выбирают вариант ответа. Сильные открытые VLM либо большие (7B+ параметров, нужны серверные GPU), либо заметно хуже работают на русском. Открытые русскоязычные модели VK обучались на 1×A100 (llava-gemma-2b-lora, 3 дня) и 8×A100 (llava-saiga-8b, 3–4 дня).

Компактная модель, которая хорошо понимает русский и работает на потребительской видеокарте, полезна там, где изображения нельзя отправлять в облако или где важна стоимость инференса: описание фото для незрячих и слабовидящих пользователей, предразметка и модерация пользовательского контента, поиск и атрибуты товаров по фото, ассистенты внутри приложений.

Цель

Получить модель не больше 3B параметров, которая на бенчмарках VK **GQA-ru** и **MMBench-ru** превосходит открытые модели VK, используя для обучения **только открытые данные VK** и **одну GPU с 8 ГБ памяти**.

Задачи

1. Изучить датасеты коллекции deepvk и протокол оценки lmms-eval, воспроизвести его в своём скрипте.
2. Выбрать базовую модель и измерить её качество без дообучения (zero-shot).
3. Собрать обучающую смесь из данных VK (GQA-ru train, LLaVA-Instruct-ru) с аугментацией под формат MMBench.
4. Дообучить LoRA-адаптер в пределах 8 ГБ видеопамяти.
5. Оценить модель на GQA-ru (testdev) и MMBench-ru (dev), сравнить с бейзлайнами, разобрать ошибки.
6. Подготовить демо (Gradio), отчёт и презентацию; выложить код и адаптер.

Ожидаемые результаты

- GQA-ru ExactMatch выше 51.44, лучшего результата среди моделей VK (llava-saiga-8b).
- MMBench-ru не ниже исходной модели: дообучение не должно ломать общие навыки.
- LoRA-адаптер размером в десятки мегабайт и воспроизводимый код обучения и оценки.
- Интерактивное демо: загрузить картинку и задать вопрос по-русски.

2. Данные VK и как они использовались

Все обучающие и оценочные данные взяты из открытой коллекции deepvk «Vision-Language Modeling» на Hugging Face. Это переводы классических англоязычных наборов на русский, сделанные с помощью GPT-4 и с ручной проверкой частых ошибок, плюс инструкции, сгенерированные GPT-3.5 по схеме LLaVA.

Датасет	Объём	Роль в проекте	Что взяли
deepvk/GQA-ru, train_balanced	40 000 вопросов / 27 519 картинок	обучение + hold-out	6 000 картинок → 8730 вопросов; ещё 300 картинок (437 вопросов) отложены для проверки
deepvk/GQA-ru, testdev_balanced	12 216 вопросов / 398 картинок	бенчмарк	целиком, только для итоговой оценки
deepvk/MMBench-ru, dev	3 910 записей (1 109 вопросов)	бенчмарк	целиком, только для оценки (обучающей части нет)
deepvk/LLaVA-Instruct-ru	144 тыс. диалогов (COCO)	регуляризация	800 коротких диалогов, картинки скачаны из COCO

Из этих источников собрана смесь трёх типов примеров:

- **gqa (8730)** — вопрос и пост-промпт «Ответь одним словом.» → короткий ответ. Это формат бенчмарка GQA-ru и обучения моделей VK.
- **gqa_mc (2000)** — открытые вопросы GQA-ru, превращённые в вопросы с 4 вариантами в формате MMBench («Options: A. … D.», «Выбери правильный вариант ответа буквой.») → буква. Дистракторы — частые ответы на вопросы того же детального типа (`types.detailed`), например другие предметы мебели для вопроса о мебели. Почти бинарные типы (слева/справа) и словоформы правильного ответа исключены. Под MMBench-ru нет обучающих данных, поэтому такая аугментация учит формату множественного выбора на русском, не используя ни одного вопроса из бенчмарка.

- **llava (800)** — 1–2 реплики из LLaVA-Instruct-ru с развёрнутыми ответами. Нужны, чтобы модель не разучилась отвечать подробно: в демо она должна уметь не только отвечать одним словом.

Hold-out из картинок train, не вошедших в обучение, нужен, чтобы проверять качество, не заглядывая в testdev. Гиперпараметры не подбирались по бенчмаркам: на всё был один прогон.

3. Метод

Выбор базовой модели

Модели VK построены по архитектуре LLaVA-1.5: CLIP ViT-L/336 + MLP-проектор + LLM (Gemma-2B или Llama-3-8B). Современные компактные VLM устроены так же (визуальный энкодер → проектор → LLM), но предобучены на порядки большем объёме мультимодальных данных и поддерживают произвольное разрешение изображений. Выбрана **Qwen3-VL-2B-Instruct** (Apache-2.0): многоязычная, нативно поддерживается в transformers 5, помещается в 8 ГБ в bf16. Более новая Qwen3.5-2B отклонена: её линейное внимание (Gated DeltaNet) без специальных ядер медленно работает на Windows.

Изображение режется на патчи 16×16 px, соседние 2×2 патча сливаются в один токен. Картинка GQA (~500×375) ограничена 448×448 px, что даёт до 196 визуальных токенов. Это ключевое условие для обучения в 8 ГБ.

LoRA

LoRA (Low-Rank Adaptation) замораживает исходные веса W и обучает добавку низкого ранга: $W' = W + (\alpha/r)\cdot B\cdot A$, где $A \in R^{r\times d}$, $B \in R^{d\times r}$, $r \ll d$. Адаптеры добавлены во все линейные слои языковой модели (q, k, v, o, gate, up, down — 28 слоёв). Визуальный энкодер и проектор заморожены: их предобучение на порядки больше наших данных, а «учить» нужно формат и стиль русских ответов.

Параметр	Значение
Базовая модель	Qwen/Qwen3-VL-2B-Instruct, bf16, SDPA-внимание
LoRA	r = 16, α = 32, dropout 0.05, все линейные слои LLM
Обучаемых параметров	17.4 млн (0.81% модели)
Данные	11530 примеров, 1 эпоха
Батч	4 × накопление 4 = 16 примеров на шаг, шагов оптимизатора: 721
Оптимизатор	AdamW, lr 0.0002, косинусное расписание, warmup 3%, clip 1.0
Лосс	кросс-энтропия только по токенам ответов ассистента
Память	gradient checkpointing; пик 7.34 ГБ
Время	37.7 мин на RTX 5060 Laptop 8 ГБ



Рис. 1. Функция потерь на обучающей смеси.

Инженерные детали

- Свой цикл обучения (~170 строк) вместо Trainer: прозрачная маскировка лосса, логирование в CSV.
- **Логиты только на позициях ответа.** Первый запуск упал с CUDA OOM на ~95-м шаге: при словаре в 152 тыс. токенов логиты всей последовательности (fp32 + градиент) занимали несколько гигабайт, хотя лосс нужен только на 2–4 токенах ответа из ~250. Передаём в модель `logits_to_keep` с индексами позиций ответа и считаем кросс-энтропию сами. Лосс на первом шаге совпал до 4-го знака (1.0868). На 90-м шаге пик памяти — 6.6 ГБ вместо 7.8 (максимум за весь прогон — 7.34 ГБ), скорость выросла с 2.5 до 4.9 примера/с.
- Hugging Face Hub отдавал ~0.4 МБ/с на соединение. Написан загрузчик с HTTP Range в 32 потока (`src/download.py`), который даёт ~8 МБ/с: 8 ГБ моделей и данных скачиваются за ~15 минут.
- Из parquet с картинками GQA-ru (3.9 ГБ) по row group извлекаются только нужные картинки.
- Оценка батчевая (16 вопросов), с левым паддингом и докачкой прерванных прогонов.

4. Протокол оценки

Чтобы цифры были сопоставимы с опубликованными метриками VK, оценка повторяет конфигурацию задач lmms-eval `gqa-ru` и `mmbench_ru_dev`:

- **GQA-ru** — testdev_balanced, 12 216 вопросов; пост-промпт «\nОтветь одним словом.»; жадное декодирование, до 16 токенов; **ExactMatch** без учёта регистра и пунктуации. Дополнительно считается мягкая метрика (ё→е, числа→слова), но основная — строгая.
- **MMBench-ru** — dev, 3 910 записей. Это 1 109 вопросов, каждый в 2–4 вариантах с циклически переставленными ответами. Промпт «{hint} {question} Options:\nA. ...» + «\nВыбери правильный вариант ответа буквой.». Основная метрика — **CircularEval**, как в MMBench/lmms-eval: вопрос засчитывается, только если верно отвечены все его перестановки. Буква извлекается правилами, без GPT-судьи (как и у VK). Дополнительно приводится обычная точность по записям.

> Оговорка: метрики моделей VK посчитаны через lmms-eval, наши — собственной реализацией того же протокола. Для GQA-ru проверено, что наша нормализация даёт тот же результат, что строгий exact_match (HF evaluate, используется в lmms-eval), — совпадение до сотых. Для MMBench-ru буква извлекается собственными правилами, здесь небольшие расхождения с lmms-eval возможны. Перепроверить модели VK нашим скриптом не позволили сроки: веса llava-gemma-2b-lora хранятся в fp32 (12 ГБ).

5. Результаты

Модель	GQA-ru EM	GQA-ru мягкая	MMBench-ru Circular	MMBench-ru по записям
deepvk/llava-gemma-2b-lora (3B)	46.4	—	40.2	—
deepvk/llava-saiga-8b (8B)	51.4	—	56.7	—
Qwen3-VL-2B, без дообучения	41.1	41.3	66.5	74.6
Qwen3-VL-2B + LoRA (наша)	**52.5**	**52.7**	**68.7**	**77.9**

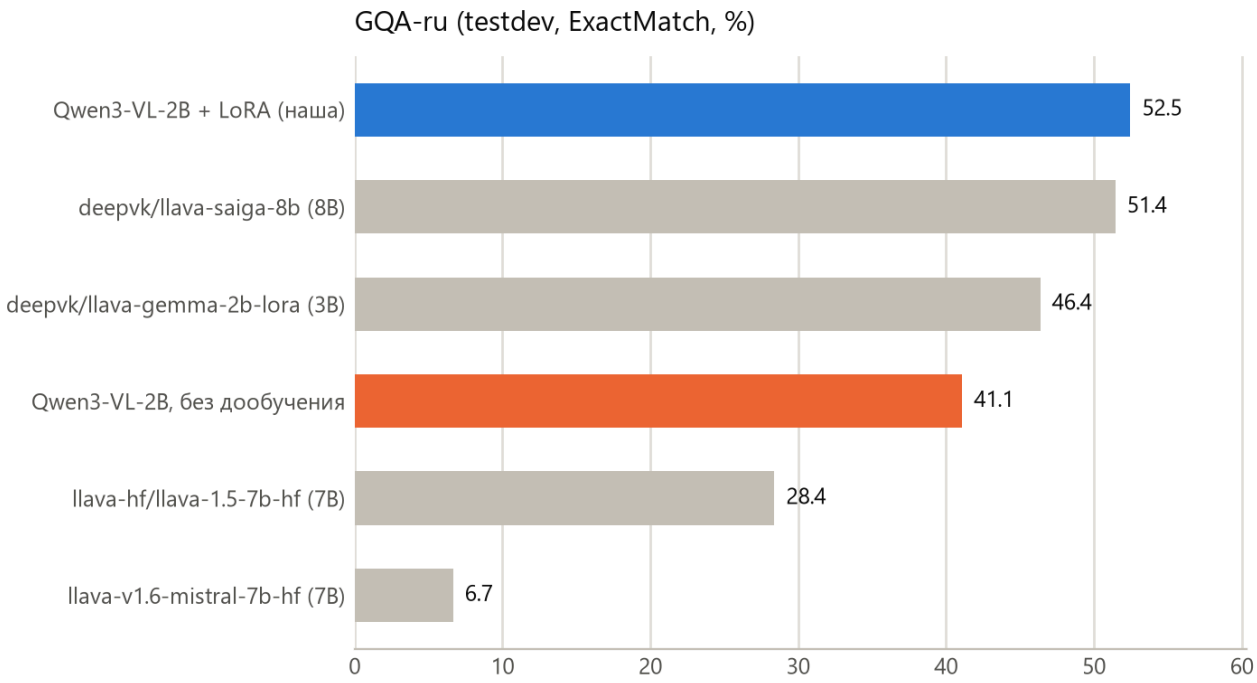


Рис. 2. GQA-ru testdev, ExactMatch. Серым — опубликованные модели VK и LLaVA.

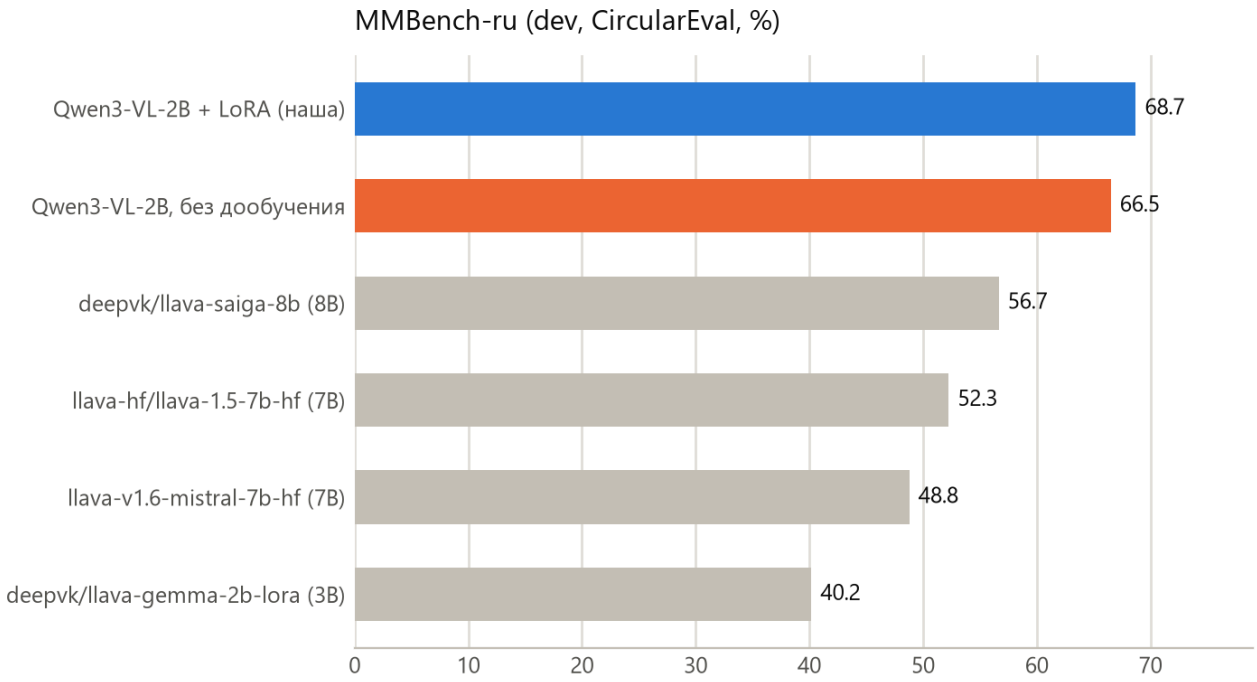


Рис. 3. MMBench-ru dev, CircularEval.

> Оговорка по MMBench-ru. У исходной модели 6.0% ответов не удалось разобрать: вместо буквы она начинает рассуждать («Рассмотрим расположение магнитов…»), и ответ обрезается лимитом в 32 токена. У дообученной модели таких ответов 0%. Если засчитать все нераспознанные ответы исходной модели как верные, её верхняя граница — 72.1 (CircularEval). Поэтому разница 66.5 → 68.7 — это в основном выученный формат ответа (эффект MC-аугментации), а не новые знания. Корректный вывод: дообучение под GQA-ru не испортило общие навыки, и обе версии Qwen3-VL-2B заметно опережают модели VK (56.7). Проверить исходную модель с длинной генерацией (до 512 токенов) мы пытались, но на 8 ГБ это шло ~15 минут на батч, и проверку пришлось остановить.

На отложенной выборке (hold-out, 437 вопросов к 300 картинкам train, не попавшим в обучение) ExactMatch вырос с 43.0 до 60.0 (+16.9 п.п.). Это больше, чем на testdev (+11.4): hold-out взят из того же распределения train_balanced, что и обучение, а testdev — отдельная выборка. Значит, часть выученного специфична для распределения train. Выборка hold-out небольшая, погрешность порядка ±2–3 п.п.

Разбивка по типам вопросов GQA-ru

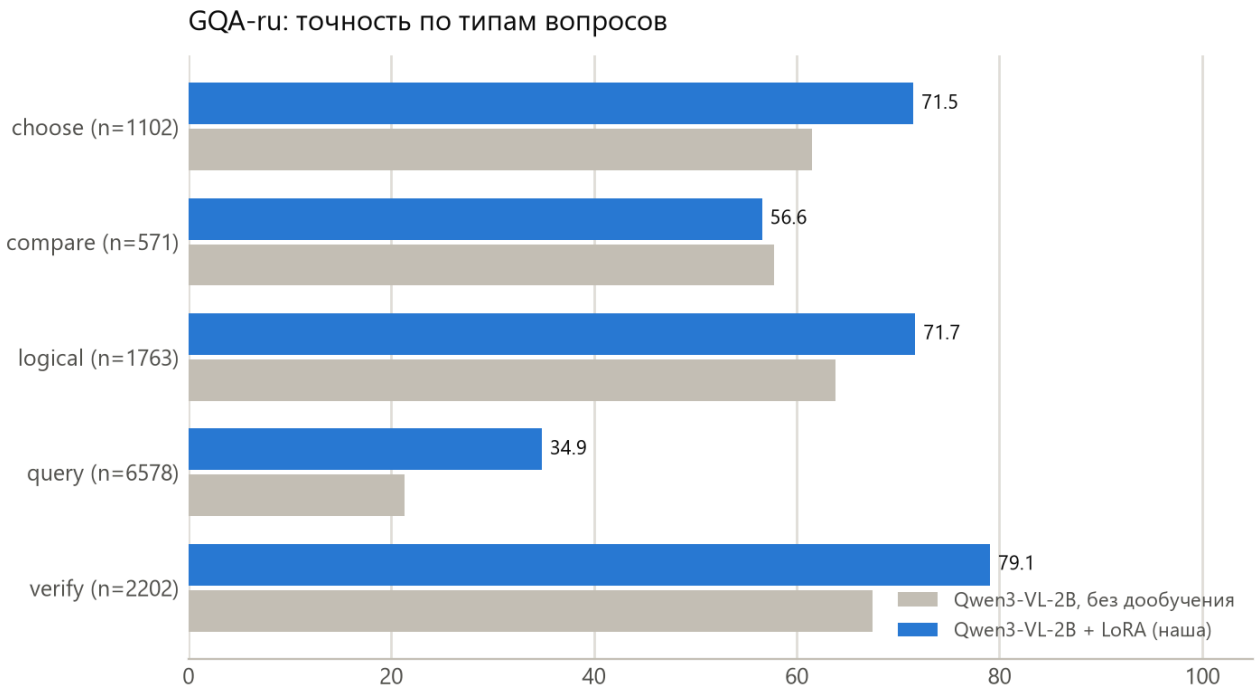


Рис. 4. ExactMatch по структурному типу вопроса: verify — да/нет, query — открытый вопрос, choose — выбор из двух названных, logical — и/или, compare — сравнение.

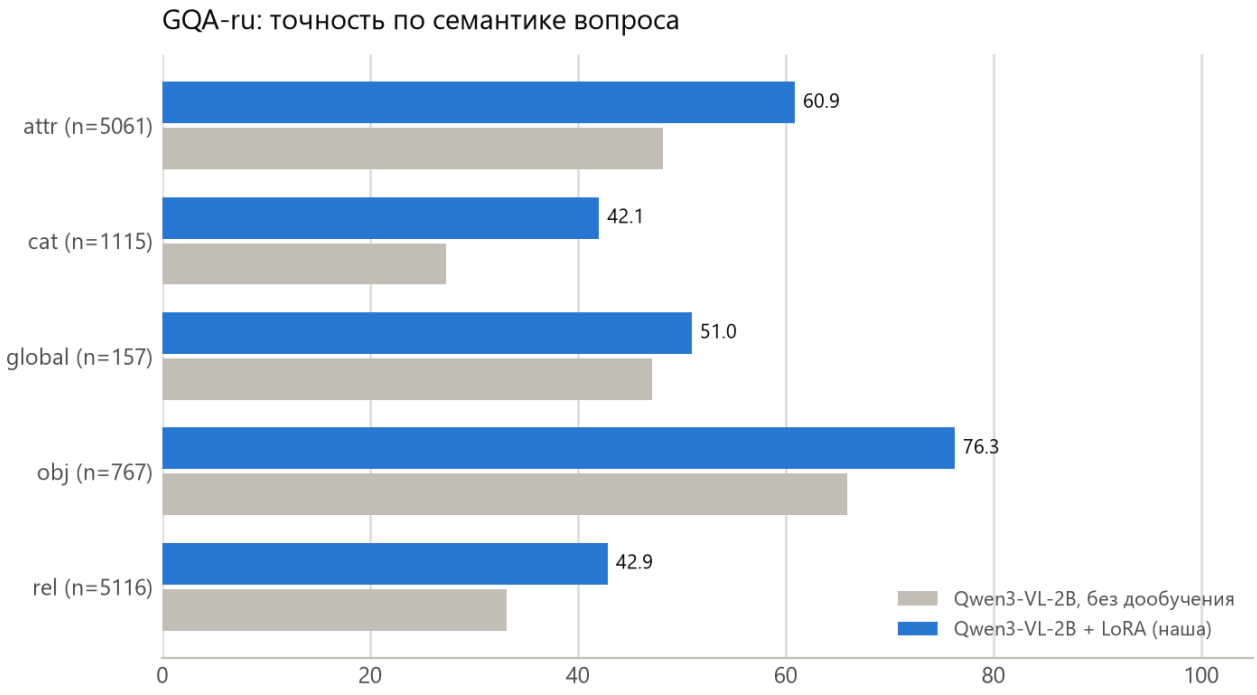


Рис. 5. ExactMatch по семантике вопроса: rel — отношения объектов, attr — атрибуты, obj — наличие объекта, cat — категория, global — сцена целиком.

MMBench-ru по группам навыков

Группа (L2)	Без дообучения	LoRA	Δ	Вопросов
attribute_reasoning	66.5	71.9	+5.4	185
coarse_perception	80.9	79.5	−1.4	288
finegrained_perception (cross-instance)	57.8	61.7	+3.9	128
finegrained_perception (instance-level)	65.5	69.4	+3.9	255
logic_reasoning	24.1	34.2	+10.1	79
relation_reasoning	71.8	66.4	−5.5	110

6. Анализ: что изменило дообучение

Из 12216 вопросов GQA-ru дообучение исправило 2227 ответов и испортило 838. Обе модели отвечают одним словом (в среднем 1.05 слова), так что дело не в длине ответа. Прирост дают два эффекта.

1. Калибровка «да/нет». Исходная модель на русском систематически склоняется к «нет»: 2944 ответов «нет» против 1378 «да», хотя в эталоне 2143 и 2277. Самая частая её ошибка — «да → нет» (в 1133 вопросах). После LoRA распределение 2407 / 2007, а точность на вопросах «да/нет» выросла с 65.3 до 73.9.

2. Форма и лексика открытых ответов. Эталоны GQA-ru — результат перевода с устойчивыми конвенциями: наречие вместо прилагательного («слева», а не «левая»), предлог в ответах на «на чём» («на столе»), согласование рода («белая»), конкретная лексика. Исходная модель часто отвечает по смыслу верно, но в другой форме, а иногда по-английски («слоны → elephant»). Точность на открытых вопросах выросла с 27.4 до 40.3.

Частые ошибки в формате «эталон → ответ модели (число случаев)»:

Без дообучения	После LoRA
да → нет (1133)	нет → да (641)
нет → да (303)	да → нет (508)
слева → левая (122)	справа → слева (52)
справа → правая (111)	девочка → женщина (42)
стул → стол (41)	автомобиль → машина (42)
слева → справа (38)	письменный стол → стол (35)
письменный стол → стол (36)	кот → кошка (35)
женщина → девушка (32)	стул → стол (35)

Примеры исправленных ответов:

Вопрос	Эталон	Без дообучения	LoRA
Какого цвета дом, коричневый или белый?	коричневый	белый	коричневый
Это банан или груша?	груша	яблоко	груша
Какого размера кровать?	большая	120x180	большая
На какой стороне фотографии находится молодой серфингист?	справа	правая	справа
Какое животное сильное?	слон	elephant	слон
На какой стороне белое полотенце?	слева	левая	слева
Холодильник находится слева от серебристой вещи, которая находится слева от хлеба?	да	фирменный	да
Какого цвета, по-вашему, ботинки, которые идут по траве?	зелёные	зеленый	зелёные
Вы видите мотоциклы или флаги?	да	нет	да
С какой стороны изображения находится вилка?	справа	правая	справа

Примеры ухудшений (случайная выборка):

Вопрос	Эталон	Без дообучения	LoRA
Пирс сделан из того же материала, что и самолёт?	нет	нет	да
Кто носит обувь?	мужчина	мужчина	женщина
Человек рядом с джинсами молодой или старый?	старый	старый	молодой
Какое животное сидит на стуле?	кот	кот	кошка
Есть ли забор перед деревьями?	нет	нет	да
Из чего сделаны столовые приборы?	дерево	дерево	металл

Что осталось. Больше всего ошибок по-прежнему в «да/нет» на сложных пространственных отношениях, теперь с лёгким перекосом в сторону «да». Вторая группа — синонимы, которые ExactMatch не засчитывает («машина» вместо «автомобиль», «кошка» вместо «кот», «женщина» вместо «девочка»): часть этих ответов по смыслу верна. Третья — путаница «слева/справа». Единственный тип вопросов без прироста — compare (сравнение двух объектов): 57.8 → 56.6.

7. Практическая применимость

- **Размер.** Базовая модель — 4.3 ГБ (bf16), адаптер — 70 МБ. Можно хранить один базовый чекпойнт и несколько адаптеров под разные задачи.
- **Железо.** Веса занимают 3.97 ГБ, пик при батче из 16 вопросов — 4.66 ГБ видеопамяти, то есть хватает потребительской GPU 8 ГБ. Короткий ответ на один вопрос — 0.21 с, описание изображения на 128 токенов — 3.2 с, батчевая обработка — 13.8 вопроса/с (RTX 5060 Laptop).

- **Стоимость обучения.** 38 минут на ноутбуке против 3 дней на A100 (llava-gemma-2b-lora) и 3–4 дней на 8×A100 (llava-saiga-8b).
- **Развёрнутые ответы сохранились.** Пример описания без пост-промпта: «На фотографии изображен сноубордист, который прыгает в воздухе. Он одет в черную куртку и зеленые брюки, а также носит защитные очки.»
- **Приватность.** Модель работает локально, изображения не покидают устройство.
- **Режимы ответа.** Короткий ответ («Ответь одним словом.») для разметки и поиска, развёрнутый — для ассистента и описания изображений (см. демо `src/demo.py`).

8. Ограничения и дальнейшая работа

- Сравнение с моделями VK опирается на их опубликованные цифры; реализации оценки близки, но не идентичны.
- Прирост на GQA-ru в основном объясняется калибровкой ответов «да/нет» и усвоением формы ответов датасета. Метрика выросла честно, но рост визуального «понимания» отдельно не измерялся.
- MMBench-ru не имеет обучающей части; синтетические вопросы с вариантами из GQA покрывают только часть навыков (нет OCR, научных диаграмм, рассуждений над текстом).
- Из-за сроков: один прогон, без подбора гиперпараметров и без абляций по составу смеси.
- Дальше: обучение на всём GQA-ru train (40 тыс. вопросов), больший вклад LLaVA-Instruct-ru, модель на 4B (Qwen3-VL-4B в 4-битной QLoRA), абляции по доле MC-аугментации.

9. Воспроизведение

Весь код, обученный адаптер, предсказания и метрики лежат в репозитории **github.com/geozasimov/vk_vlm_project**.

Шаг	Команда
Окружение	`pip install torch==2.11.0 torchvision==0.26.0 --index-url https://download.pytorch.org/whl/cu128` и `pip install -r requirements.txt`
Загрузка	`python src/download.py model gqa-train llava`
Данные	`python src/prepare_data.py --n-images 6000 --n-mc 2000 --n-llava 800`
Обучение + оценка	`bash run_pipeline.sh --epochs 1 --batch-size 4 --grad-accum 4`
Отдельно	`python src/eval_gqa.py --tag lora --adapter outputs/lora-adapter`; `python src/eval_mmbench.py --tag lora --adapter outputs/lora-adapter`
Демо	`python src/demo.py`

10. Итоги: ожидаемые и достигнутые результаты

Ожидали	Получили	Статус
GQA-ru выше 51.44 (llava-saiga-8b)	52.5 ExactMatch	выполнено
MMBench-ru не ниже исходной модели	68.7 против 66.5 (верхняя граница исходной — 72.1)	выполнено, прирост — в основном формат
Адаптер размером в десятки МБ	70 МБ, 17.4 млн параметров	выполнено
Обучение на одной GPU 8 ГБ	38 мин, пик 7.34 ГБ	выполнено
Интерактивное демо	Gradio, короткий и развёрнутый режимы	выполнено

Гипотеза подтвердилась: современная компактная VLM, дообученная только на открытых данных VK за 38 минут на ноутбуке, превосходит открытые модели VK на обоих бенчмарках, включая 8B-модель, обученную на кластере. Основной вклад в рост на GQA-ru дали калибровка ответов «да/нет» и усвоение формы ответов датасета. На MMBench-ru дообучение не навредило общим навыкам и закрепило формат ответа буквой.

Источники

- Коллекция deepvk Vision-Language Modeling: huggingface.co/collections/deepvk/vision-language-modeling-664dd7e4c257cc78e740f6bc
- Vision Language Models Explained: huggingface.co/blog/vlms
- Карточки моделей deepvk/llava-gemma-2b-lora и deepvk/llava-saiga-8b (таблицы метрик).
- Qwen3-VL: huggingface.co/Qwen/Qwen3-VL-2B-Instruct
- Hu et al. LoRA: Low-Rank Adaptation of Large Language Models, 2021.
- Hudson, Manning. GQA, CVPR 2019; Liu et al. MMBench, 2023; Liu et al. Visual Instruction Tuning, 2023.
- lmms-eval: github.com/EvolvingLMMs-Lab/lmms-eval